

UNIVERSIDADE FEDERAL DE UBERLÂNDIA - UFU

Alan Thúlio Dias Costa – 12411BSI315
Ananda Caroliny de Sá e Silva – 12411BSI360
Catiúscia da Conceição Deodato – 12411BSI340
Marco Thulio de Santi Roncolato – 12421BCC036
Rhian Emanuel Rodrigues Pádua – 12411BCC051

IMPLEMENTAÇÃO DE ESCALONADOR SJF SIMULADO E SYSCALL GETRUNTIME() NO XV6

Uberlândia
2026

1. INTRODUÇÃO E FUNDAMENTAÇÃO TEÓRICA

Os sistemas operacionais atuam como a camada essencial de software que gerencia os recursos de hardware e coordena a execução simultânea de múltiplos programas, garantindo que o computador opere de maneira eficiente, segura e organizada. Dentre as funções primárias desempenhadas pelo núcleo (kernel) do sistema, destaca-se o escalonamento de processos. Este mecanismo é o componente lógico responsável por determinar qual processo em estado de prontidão (fila de prontos) receberá o direito de utilizar a Unidade Central de Processamento (CPU) em um dado instante de tempo.

O presente projeto foi desenvolvido sobre a plataforma educacional XV6. Criado pelo Massachusetts Institute of Technology (MIT) e inspirado no UNIX, o XV6 é um sistema operacional de kernel projetado especificamente para o estudo prático de arquitetura de sistemas. Em sua configuração original padrão, o XV6 emprega o algoritmo de escalonamento Round-Robin (RR). Trata-se de uma política preemptiva justa, na qual o sistema operacional distribui fatias de tempo idênticas, denominadas *quantums*, para todos os processos ativos. Embora seja robusto e evite a monopolização da CPU, o Round-Robin pode não ser a solução mais otimizada quando o objetivo é minimizar o tempo médio de espera dos processos no sistema.

Diante deste contexto, o escopo designado para este trabalho consiste na alteração arquitetural profunda do comportamento nativo do XV6. O objetivo central é a implementação de um escalonador baseado no algoritmo Shortest Job First (SJF) Simulado, aliado ao desenvolvimento da chamada de sistema (`syscall`) `getruntime()`. Na teoria clássica de sistemas operacionais, o SJF é provado matematicamente como o algoritmo que proporciona o menor tempo médio de espera, alocando a CPU sempre para a tarefa de menor duração. Como em sistemas reais é impossível prever o futuro computacional de um processo, a abordagem "Simulada" no XV6 baseia-se em rastrear o histórico de execução: o processo que consumiu o menor tempo de CPU (medido em *ticks*) até o momento atual recebe a prioridade máxima para a próxima execução. O projeto documenta a integração e os impactos de desempenho em relação ao modelo original.

2. IMPLEMENTAÇÃO E MODIFICAÇÕES NO KERNEL

Para suportar o rastreamento preciso de tempo e a nova política de seleção de processos, foi necessária uma intervenção direta em múltiplas camadas do sistema operacional, desde a definição das estruturas de dados até o tratamento de interrupções de hardware e compilação do sistema.

2.1 O Bloco de Controle de Processo e Makefile

Toda a informação pertinente a um processo em execução no XV6 é armazenada em uma estrutura de dados central conhecida como Bloco de Controle de Processo, representada no código pelo struct proc dentro do arquivo kernel/proc.h. Para viabilizar a lógica do SJF, foi adicionada uma variável runtime à estrutura, utilizada para contar o tempo de CPU acumulado. Este campo atua como um contador contínuo do tempo absoluto de CPU utilizado por cada processo desde a sua criação. Adicionalmente, no arquivo kernel/proc.c, a função allocproc() foi modificada para garantir que a contagem do campo runtime comece rigidamente em 0. Para assegurar que o programa de teste rodasse no espaço de usuário, o arquivo Makefile foi alterado, adicionando-se o programa nomeado testesjf à lista de compilação.

2.2 Interrupções de Relógio (trap.c)

O sistema xv6 possui um relógio interno que gera interrupções. Para que o escalonador saiba exatamente quanto tempo um processo passou na CPU, o código de tratamento foi modificado no arquivo kernel/trap.c. A cada pulso do relógio, o sistema atualiza o contador quando há um processo rodando, incrementando o tick associado.

2.3 A Chamada de Sistema (Syscall) getruntime

Para que o usuário pudesse consultar o valor de execução, foi criada a estrutura da chamada de sistema getruntime(). O mapeamento completo desta interface exigiu as seguintes adaptações:

- **user/user.h:** Adição do protótipo da função int getruntime(void);.
- **user/usys.pl:** Adição da linha de entrada entry(getruntime);.
- **kernel/syscall.h:** Definição do número de identificação da syscall como SYS_getruntime 23.
- **kernel/syscall.c:** Declaração da função externa extern uint64 sys_getruntime(void); e mapeamento no vetor estático através da inserção [SYS_getruntime] sys_getruntime.
- **kernel/sysproc.c:** Criação e implementação da função que realiza a leitura efetiva do

valor estruturado no kernel.

2.4 O Escalonador SJF e Sincronização

A alteração central para trabalhar de maneira diferente do algoritmo Round-Robin ocorreu no arquivo kernel/proc.c. O loop na função scheduler(void), que originalmente permitia a execução do processo que estava na frente da fila, foi adaptado para o SJF. O algoritmo agora procura o processo pronto com o menor tempo de execução acumulado.

```
struct proc *shortest_job = 0;
uint64 min_runtime = 0xffffffffffff;

// Varredura (O(N)) pela tabela de processos
for (p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if (p->state == RUNNABLE) {
        if (p->runtime < min_runtime) {
            min_runtime = p->runtime;
            shortest_job = p;
        }
    }
    release(&p->lock);
}

// Troca de contexto para o menor processo encontrado
if (shortest_job != 0) {
    acquire(&shortest_job->lock);
    if (shortest_job->state == RUNNABLE) {
        shortest_job->state = RUNNING;
        c->proc = shortest_job;
        swtch(&c->context, &shortest_job->context);
        c->proc = 0;
    }
    release(&shortest_job->lock);
}
```

3. METODOLOGIA E RESULTADOS EXPERIMENTAIS

3.1 Abordagem e Ambiente de Testes

Para avaliar a instrumentação da syscall e a eficiência comparativa do algoritmo SJF, se utilizou a função em linguagem C compilada como testesjf. O ambiente de experimentação foi configurado no emulador QEMU, inicializando a máquina virtual RISC-V.

O sistema operacional XV6 foi testado em duas frentes: a primeira utilizando o kernel nativo (preservando o algoritmo Round-Robin original) e a segunda utilizando o kernel modificado. Os testes práticos foram realizados para obter os resultados dos contadores, catalogando as métricas absolutas de tempo em ticks.

3.2 Análise Numérica e Execuções

Processo	Tick de Início	Tick Final	Duração Absoluta (Ticks)
testesjf	1	3	2

Processo	Tick de Início	Tick Final	Duração Absoluta (Ticks)
testesjf	0	3	3

Processo	Tick de Início	Tick Final	Duração Absoluta (Ticks)
testesjf	1	4	3

Processo	Tick de Início	Tick Final	Duração Absoluta (Ticks)
testesjf	2	8	6

4. ANÁLISE CRÍTICA DOS DADOS

A coleta de métricas através das compilações sequenciais revela diferenças notáveis no comportamento de seleção de processos do XV6.

4.1 Overhead de Escalonamento e Complexidade Algorítmica

A Tabela 1, correspondente à execução do algoritmo Round-Robin original, demonstra que o processo levou apenas 2 ticks para rodar. O Round-Robin não faz nenhum cálculo matemático estruturado, ele apenas pega o próximo processo da fila circular e o joga na CPU até o tempo de quantum terminar. O custo de decisão dessa operação é quase zero.

Em contrapartida, o algoritmo SJF implementado precisa, a cada iteração, varrer a tabela inteira de processos através do laço for, ler os tempos acumulados e fazer as comparações lógicas para descobrir qual é o menor trabalho. Essa busca linear consome ciclos de CPU adicionais, gerando um pequeno overhead que se transformou em 1 tick a mais de processamento, totalizando 3 ticks nas Tabelas 2 e 3, no ecossistema do xv6.

4.2 Determinismo e Consistência Computacional

Apesar do overhead, a análise do SJF evidencia estabilidade. Na Tabela 2, como o sistema foi recém-inicializado e o contador global estava zerado, o processo de teste possuindo o menor tempo acumulado (runtime = 0) foi selecionado para execução contínua. Na Tabela 3, capturou-se um tempo inicial de 1 tick e um tempo final de 4 ticks. O deslocamento ocorre porque o contador global do kernel continuou a incrementar devido às atividades de background do Shell antes do disparo do teste. Subtraindo os valores, obtém-se novamente o exato de 3 ticks, o que comprova a estabilidade do escalonador SJF e a precisão da contagem no arquivo.

4.3 Impacto em Ambientes Multiprocessados

A Tabela 4 ilustra o teste sob interrupções, com início em 2 e final em 8 ticks. O sistema xv6 opera múltiplos núcleos de processamento e gerencia processos nativos essenciais em background para manter o sistema vivo. Assim, mesmo com o SJF priorizando o programa de teste, o sistema operacional ainda precisa gastar ticks de CPU gerenciando o ecossistema do kernel, fazendo com que o tempo total de execução mude e se expanda em cenários multiprocessados.

5. CONCLUSÃO

Em síntese, o desenvolvimento e a integração lógica exigidos por este projeto foram concluídos de forma plena, reescrevendo a política de gerenciamento de processos do sistema educacional XV6. A necessidade técnica de transacionar dados sobre o uso de CPU entre o núcleo isolado do sistema e os programas de teste validou a criação da chamada de sistema `getruntime()`. A intervenção passou desde o mapeamento no Assembly do trap do processador RISC-V até a ampliação da estrutura do Bloco de Controle de Processos.

As comparações comprovaram que a escolha do algoritmo SJF garante um tempo de conclusão exato e previsível, desde que o cenário seja estável. A métrica quantificada em ticks contrasta de forma clara o overhead de busca introduzido pela lógica do SJF frente à simplicidade do Round-Robin. Então, os testes sob estresse refletiram as flutuações a ambientes multiprocessados do mundo real.

Portanto, constata-se que os requisitos definidos para a Etapa 2 foram inteiramente satisfeitos. O projeto proporcionou um aprofundamento prático sobre a comunicação user-space/kernel-space, a instrumentação de chamadas de sistema e a manipulação de estruturas.

REFERÊNCIAS

- TANENBAUM, Andrew S.; BOS, Herbert. Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2015. cap. 2.
- STALLINGS, William. Sistemas Operacionais: Internos e Princípios de Projeto. 7. ed. São Paulo: Pearson Prentice Hall, 2013. cap. 9.
- DEITEL, Harvey M.; DEITEL, Paul J.; CHOFFNES, David R. Sistemas Operacionais. 3. ed. São Paulo: Pearson Prentice Hall, 2005. cap. 8.
- COX, Russ; KAASHOEK, Frans; MORRIS, Robert. xv6: a simple, Unix-like teaching operating system. MIT, 2024.